

About Storage in JavaScript

Professional Developer Documentation

Comprehensive Guide to localStorage, sessionStorage and Cookies

Prepared for Frontend Development & Interview Preparation

Table of Contents

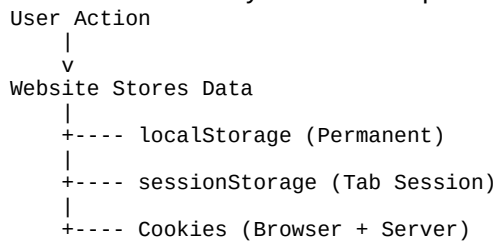
1. Introduction
2. Why Storage is Needed
3. localStorage
4. sessionStorage
5. Cookies
6. Comparison Table
7. Architecture Diagrams
8. Security & Best Practices
9. Interview Questions
10. Quick Revision Sheet

1. Introduction

Modern web applications need mechanisms to persist information between page refreshes, browser sessions and server requests. JavaScript provides three primary storage options: `localStorage`, `sessionStorage` and `Cookies`. Each serves a different purpose and understanding their strengths and limitations is essential for building production-grade applications.

2. Why Storage is Needed

Without storage, all application state would disappear whenever the page refreshes. Storage enables login persistence, shopping carts, dark mode preferences, language settings, multi-step forms, dashboards and many other user experiences.



3. `localStorage`

`localStorage` stores data permanently inside the browser until explicitly removed.

Common Use Cases

- Dark mode preference
- Language preference
- Shopping cart persistence
- Saved user settings
- Draft content recovery

Example

```
// Store
localStorage.setItem("theme", "dark");

// Read
const theme = localStorage.getItem("theme");

// Delete
localStorage.removeItem("theme");

// Clear everything
localStorage.clear();
```

Storing Objects

```
const user = {
  name: "Shivam",
  role: "Engineer"
};

localStorage.setItem(
  "user",
  JSON.stringify(user)
);

const data = JSON.parse(
  localStorage.getItem("user")
);
```

4. sessionStorage

sessionStorage behaves similarly to localStorage but exists only for the current tab session.

Use Cases

OTP verification flows, checkout processes, multi-step forms and temporary application state.

```
sessionStorage.setItem("step", "2");
```

```
const currentStep =  
sessionStorage.getItem("step");
```

5. Cookies

Cookies are small pieces of data automatically sent with HTTP requests.

```
// Create Cookie  
document.cookie = "user=Shivam";  
  
// Cookie with Expiry  
document.cookie =  
"user=Shivam; expires=Fri, 30 Dec 2027 12:00:00 UTC";
```

Authentication Flow

Server ---> Set-Cookie: token=abc123

Browser Stores Cookie

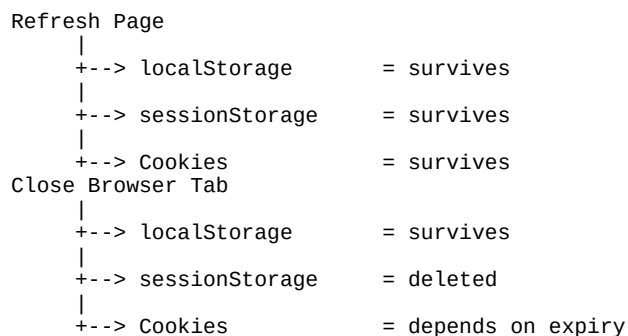
Future Requests:
Cookie: token=abc123

Server identifies user

6. Comparison Table

Feature	localStorage	sessionStorage	Cookies
Persistence	Permanent	Until Tab Closes	Configurable
Size	~5-10 MB	~5-10 MB	~4 KB
Sent to Server	No	No	Yes
Best Use	Preferences	Temporary Data	Authentication

7. Architecture Diagrams



8. Security & Best Practices

- Do not store sensitive information such as passwords.
- Prefer HttpOnly cookies for authentication tokens.
- Use Secure cookies over HTTPS.
- Validate and sanitize user input to reduce XSS risk.
- Avoid storing large datasets in cookies.
- Use JSON.stringify and JSON.parse consistently.

JWT Storage Discussion

Many beginners store JWTs in localStorage because it is easy. In production systems, HttpOnly Secure Cookies are generally preferred because JavaScript cannot directly access them, reducing the impact of XSS vulnerabilities.

9. Interview Questions & Answers

Q. Difference between localStorage and sessionStorage?

A. localStorage survives browser restarts while sessionStorage is removed when the tab closes.

Q. Which storage is automatically sent to the server?

A. Cookies.

Q. Can localStorage store objects directly?

A. No. Objects must be converted using JSON.stringify.

Q. Which storage is best for dark mode preferences?

A. localStorage.

Q. Which storage is preferred for authentication in production?

A. HttpOnly Secure Cookies.

10. Quick Revision Sheet

localStorage
- Persistent
- Not sent to server
- Great for preferences

sessionStorage
- Tab specific
- Temporary
- Great for workflows

Cookies
- Small size
- Sent to server
- Great for authentication

End of Documentation